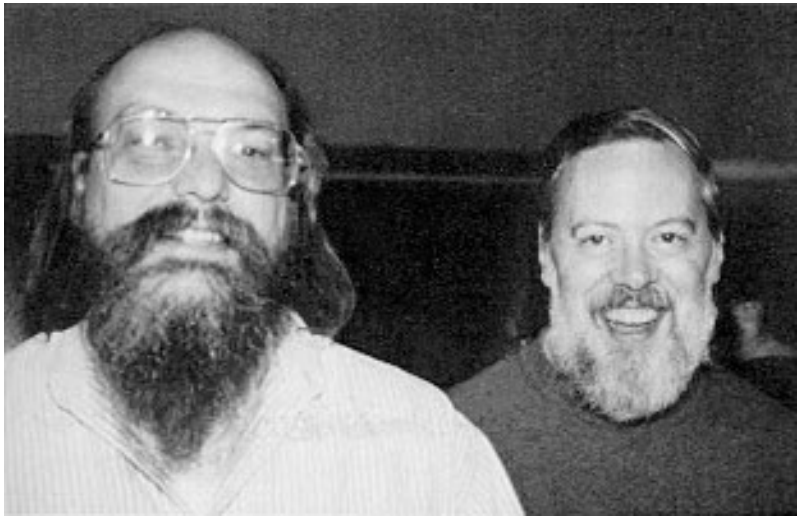


Secure Software Design and Development
(CY-321)

Reflection on Trusting Trust

Why is Trust Management So Difficult?



Code
(What you see)

Binary
(What execute)

Ken Thompson invented Unix together with Dennis Richie.

For this achievement, he was awarded the ACM Turing Award in 1984.

In his award lecture, he outlined how he modified the Unix C compiler so that he got access to any Unix system.

Reflections on Trusting Trust

He modified the system such that the compiler source code was free of any trace of malicious activity.

If the C compiler detected that the login program was compiled, it compiled in a back door that would allow Thompson access with a special user name/password combination

If the C compiler detected that it was compiling itself, it would compile in code that would create the above back door.

Now, this modification is pretty obvious, because you can see it in the C compiler source code.

What did Thompson do next?

Reflections on Trusting Trust

1. He compiled the C compiler with itself;

2. He removed the modifications from the C compiler;

3. He recompiled the C compiler with itself one more time.

That way, all traces in the source code were gone and literally no amount of source code analysis would find any problems with the compiler.

Reflections on Trusting Trust



"The moral is obvious. You cant trust code that you did not totally create yourself. (Especially code from companies that employ people like me.)"

—Ken Thompson

Why is Trust Management So Difficult?

A trust relationship is a relationship among the different participants in a software system and concerns the assumptions that those participants make about security properties of the other part.

For example, a function might assume that its inputs are shorter than some maximum length; or it might assume that its input is a valid user name.

Why is Trust Management So Difficult?

Traditionally, the responsibility of making sure that the functions assumptions are met lay with the caller of the function.

> "The strcpy() function copies the string pointed to by src (including the terminating '\0' character) to the array pointed to by dest. The strings may not overlap, and the destination string dest must be large enough to receive the copy." —strcpy(3) manual page

We can then say that the library routine trusts its caller to provide legal arguments.

An attacker is often interested in violating the assumptions that parts of a program make, because "interesting" things often happen if they are violated.

Why is Trust Management So Difficult?

Often, programmers extend trust to other parts of a program without realizing it.

Assumptions are easy to make: Object-Oriented programming has taught us that we must decompose a system into small, largely independent objects and that it is OK to forget about the big picture when we are coding individual objects.

Therefore, programmers are encouraged to think about software development in small steps.

But when they do that, they lose sight of the system as a whole and forget to make their assumptions explicit. (That happens especially with routines that are deep in the guts of a system, because the assumption is that user input will only get this far after extensive validations in the upper layers.)

Why is Trust Management So Difficult?

Its often easier to put the burden of validation on the **caller** instead of validating input in the **callee** because there is often no standard way to signal an error to the **caller**:

- the **callee** can throw an exception. This changes the control flow in a nonlinear way and often introduces objects that are not compatible with the rest of the application;
- the **callee** can return an error code. Error codes are often not appropriate for returning detailed information about the error; or
- the **callee** can set a global variable to the detailed error description and return an error value in-band. This is prone to error on multithreaded systems, besides being confusing in certain circumstances.

Input Validation

- We expect input of a certain form
  But we cannot guarantee that its safe
  - it's under the attacker's control

So we must **validate it before we trust it**

- **Making input trustworthy**
  - **Check it** has a valid form, and reject it if not
  - **Sanitize it** by modifying it or using it it in such a way that the result is correctly formed by construction

Trusting Input

Trust in input is often not warranted and sometimes downright **dangerous**.

```
#include <stdio.h>

int main() {
    int a;
    scanf("%d", &a);
    printf("%d\n", a);
    return 0;
}
```



What happens if the user enters something that is not a number?

The value of **"a"** is undefined, and therefore could be anything.

Trusting Input

```
#include <stdio.h>
#include <string.h>

int main() {
    char filename[1024];
    char command[sizeof(filename) + 4];

    fgets(filename, sizeof(filename), stdin);
    filename[sizeof(filename) - 1] = '\0';

    strcpy(command, "cat ");
    strcat(command, filename);

    system(command);
    return 0;
}
```

Why does **fgets()** limit input length but fail to prevent command injection?

use **fopen()** and **fread()**

What other problems might there be?

OS Command Injection

That is more interesting, Is there a buffer overflow? **No.**

What happens if a user enters **“ rm -rf *”?**

Trusting Input *(SQL Injection)*

```
import sqlite3

def n_rows(table, db_connection):
query_start = "SELECT COUNT(*) FROM "
    query = query_start + table

    cursor = db_connection.cursor()
    cursor.execute(query)
    result = cursor.fetchone()

    return result[0] if result else 0
```



What other problems might there be?

What if the argument isnt checked and the user can somehow enter "**customers; DROP TABLE customers**" ?

Trusting Input *(SQL Injection)*

```
import sqlite3

def get_user_data(username, db_connection):
        query = f"SELECT * FROM users WHERE username
= '{username}'"

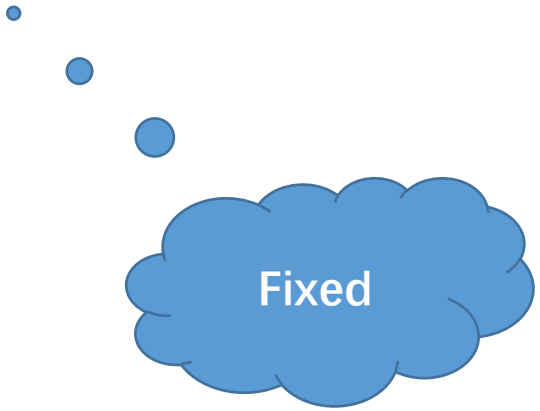
    cursor = db_connection.cursor()
    cursor.execute(query)
    return cursor.fetchall()
```

OR '1'='1' always evaluates to TRUE, meaning **all user data** will be returned.

Trusting Input

```
def get_user_data(username, db_connection):
query = "SELECT * FROM users WHERE username = ?"

    cursor = db_connection.cursor()
    cursor.execute(query, (username,))
return cursor.fetchall()
```



Trusting Input *(SQL Injection)*

```
def get_balance(account_id, db_connection):
query = f"SELECT balance FROM accounts
WHERE account_id = {account_id}"

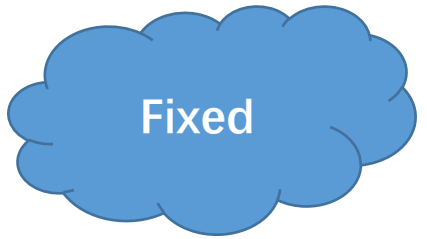
    cursor = db_connection.cursor()
    cursor.execute(query)
return cursor.fetchall()
```

What If an attacker passes **123 UNION SELECT credit_card_number FROM customers**

Trusting Input

```
def get_balance(account_id,
db_connection):
    """Retrieve account balance safely."""
    query = "SELECT balance FROM accounts
WHERE account_id = ?"

    cursor = db_connection.cursor()
    cursor.execute(query, (account_id,))
    return cursor.fetchall()
```



Trusting Input *(OS Command Injection)*

```
import os

def delete_file(filename):
    command = f"rm -rf {filename}"
    os.system(command)

delete_file("important.txt")
```

How an Attacker Exploits This

```
delete_file("important.txt; rm -rf /")
```

```
rm -rf important.txt; rm -rf /
```

Trusting Input

```
import os
import shlex

def delete_file(filename):
safe_filename = shlex.quote(filename)
os.system(f"rm -rf {safe_filename}")

delete_file("important.txt")
```

shlex.quote(filename) properly escapes shell metacharacters, preventing command injection.

Trusting Input *(OS Command Injection)*

```
import subprocess

def ping_host(host):
command = f"ping -c 4 {host}"
subprocess.Popen(command, shell=True)

ping_host("example.com")
```




```
ping_host("example.com && cat /etc/passwd")

ping -c 4 example.com && cat /etc/passwd
```

Trusting Input

```
import subprocess

def ping_host(host):
subprocess.run(["ping", "-c", "4", host],
check=True)

ping_host("example.com")
```

Uses a **list of arguments** instead of **shell=True**, preventing command injection

Trusting Input *(LDAP Injection)*

```
import ldap

username = input("Enter username: ")
password = input("Enter password: ")

# LDAP connection
conn =
ldap.initialize("ldap://example.com")
conn.simple_bind_s("cn=admin,dc=example,dc
=com", "adminpassword")

search_filter = f"(uid={username})"
result =
conn.search_s("dc=example,dc=com",
ldap.SCOPE_SUBTREE, search_filter)

if result:
    print("Authentication successful")
else:
    print("Authentication failed")
```

Use LDAP escaping functions **(ldap.filter.escape_filter_chars())**

Restrict permissions in LDAP

Implement role-based access control (RBAC)

Trusting Input *(XML Injection)*

Part-1

```
from lxml import etree

# XML Database
xml_data = """
<users>
    <user>
        <username>admin</username>
        <password>supersecret</password>
        <email>admin@example.com</email>
    </user>
    <user>
        <username>john</username>
        <password>mypassword</password>
        <email>john@example.com</email>
    </user>
</users>
"""
```

Part-2

```
tree = etree.XML(xml_data)

username = input("Enter username: ")

query =
f"//user[username='{username}']/email"
result = tree.xpath(query)

if result:
    print("Extracted Email:",
result[0].text)
else:
    print("User not found")
```

Trusting Input *(XML Injection)*




```
query = f"//user[username='{username}']/email"
```

With Attacker Input

```
query = "//user[username='admin' or contains(email, 'example.com')]/email"
```

Some Hard-And-Fast Rules

Consider all input to be untrusted and validate all user input

Encode output using the appropriate character set, escape special characters and quote input

Use structured mechanisms to separate data from code

Display generic error messages that yield minimal to no additional information.

Some Hard-And-Fast Rules

Implement fail safe by redirecting all errors to a generic error page and logging it for later review

Remove any unused functions or procedures from the database server if not needed

To mitigate OS command injection, run the code in a **sandbox** environment that enforces strict boundaries between the processes being executed and the Operating System

Some Hard-And-Fast Rules

Audit and log the queries that are executed along with their response times to detect injection attacks

Implement least privilege by using views, and restricting tables, queries and procedures to only the authorized set of users and/or accounts.

Use runtime policy enforcement to create the list of allowable commands (whitelist) and reject any command that does not match the whitelist.



Questions??

zubair.ahmad@giki.edu.pk

Office: G14 FCSE lobby